

RefactoringMiner 2.0 Output Taxonomy and Optimal Merge Tool Mapping

Architectural Taxonomy of RefactoringMiner 2.0 Outputs

The RefactoringMiner 2.0 tool produces a highly granular, threshold-free JSON output schema detailing programmatic transformations applied between repository commits. Operating directly on Abstract Syntax Trees (ASTs), it bypasses the limitations of line-based diffing and heuristic similarity thresholds to detect refactorings across six architectural tiers: packages, type declarations, methods, fields, variables, and parameters.¹ The tool identifies 40 distinct refactoring types, outputting coordinates that include file paths, line ranges, column indices, and discrete codeElementType classifications.³

To effectively resolve merge conflicts induced by these refactorings, the version control merge driver must be algorithmically paired to the specific structural tier of the RefactoringMiner 2.0 output. Traditional line-based merging (such as Git's merge-ort) inherently fails when parallel branches execute structural relocations or signature mutations, triggering false-positive conflicts.⁵ The following table establishes the optimal mapping between RefactoringMiner 2.0 output categories and the most effective research and industry merge tools based on their underlying parsing algorithms.

Code Element Level	RefactoringMiner 2.0 Output Types	Optimal Merge Tool	Algorithmic Paradigm
Package & Class	Rename Package, Change Package (Move/Split/Merge), Extract Class, Extract Subclass, Extract Superclass, Extract Interface, Move Class, Rename Class, Move and Rename Class, Collapse Hierarchy, Merge Class, Split Class	SemanticMerge (Industry) / RePatch (Research)	Location-Independent Parsing & Asymmetric Patch Transfer
Method Extractions	Extract Method, Inline Method, Extract and Move Method, Move and Inline Method, Merge Method, Split	RefMerge (Research) / Weave (Industry/CLI)	Operation-Based Reversion & Entity-Level Semantic Identity

	Method		
Signature & Relocation	Move Method, Rename Method, Pull Up Method, Push Down Method, Change Return Type, Change Method Access Modifier, Add/Remove/Reorder Parameter, Rename Parameter, Merge/Split Parameter, Change Parameter Type	Mergiraf (Industry) / Weave (Industry)	Syntax-Aware AST Matching (PCS Triples) & Structural Hashing
Field & Attribute	Move Attribute, Pull Up Attribute, Push Down Attribute, Rename Attribute, Move and Rename Attribute, Replace Variable with Attribute, Replace Attribute, Merge/Split Attribute, Change Attribute Type, Extract/Inline/Encapsulate Attribute	IntelliMerge (Research)	Graph-Based (Program Element Graphs)
Variable & Sub-Method	Extract Variable, Inline Variable, Parameterize Variable, Rename Variable, Merge/Split Variable, Change Variable Type, Split Conditional, Invert Condition, Merge Conditional, Merge Catch	Spork (Research, Java) / Mergiraf (Industry, Multi-Language)	3DM Algorithm with Formatting Preservation & GumTree Matching

Package and Class-Level Structural Transformations

When RefactoringMiner 2.0 outputs package or class-level transformations, such as Move Class, Rename Package, or Extract Class, the primary conflict mechanism revolves around inter-file structural drift.⁴ These operations physically relocate code blocks across the directory tree. Standard version control systems and even advanced file-bound AST merge tools interpret

these shifts as total file deletions on one side and raw insertions on the other.⁸ Consequently, if a parallel branch modifies a method within a class that another branch has simultaneously moved, a catastrophic merge failure occurs because the target file path no longer aligns.

Optimal Industry Solution: SemanticMerge (Plastic SCM)

For macroscopic structural shifts, SemanticMerge (integrated natively into Plastic SCM and Unity Version Control) is the premier industry tool.¹⁰ Rather than binding AST nodes to physical file paths, SemanticMerge utilizes a fully location-independent parsing engine. It constructs a virtual representation of the codebase's namespaces, classes, and methods prior to diffing.¹² When RefactoringMiner 2.0 detects a Move Class or Rename Class, SemanticMerge's Xdiff/Xmerge engine mathematically tracks the internal structure of the class across the repository boundaries.¹¹ This allows the tool to seamlessly merge intra-class logic modifications executed by Developer A into the new file pathway established by Developer B's Move Class refactoring, bypassing the traditional cross-file conflict entirely.¹³

Optimal Research Solution: RePatch

In environments maintaining long-lived forks or highly divergent variants where Rename Package or Move Class operations have compounded over time, the research tool RePatch is optimal. RePatch is an extension of the RefMerge framework designed specifically for asymmetric patch transfer.⁸ It analyzes the refactoring history to identify structural misalignments between variants. By programmatically inverting the refactorings in both the source and target repositories to realign the structural context, applying the patch, and then replaying the transformations, RePatch successfully integrates 52.8% of patches where standard Git cherry-picking fails entirely due to structural divergence.⁸

Method-Level Extractions and Inlining

Method extraction and inlining represent a unique algorithmic challenge. Outputs such as Extract Method and Inline Method indicate that a contiguous sequence of execution logic has been removed, encapsulated into a newly declared AST node, and replaced with a method invocation.¹⁶ If a concurrent branch alters the logic within the bounds of the extracted code, AST matchers frequently fail to correlate the modified base logic with the newly synthesized extracted method.¹⁷

Optimal Research Solution: RefMerge

RefMerge is specifically architected to conquer the complexities of Extract Method and Inline Method conflicts.¹⁸ Operating as an IntelliJ IDE plugin, RefMerge utilizes RefactoringMiner natively as its detection oracle. Rather than attempting to map the AST differences of the final artifacts—a process that graph-based tools like IntelliMerge explicitly struggle with—RefMerge employs an operation-based methodology.¹⁸

When an Extract Method output is detected, RefMerge programmatically interacts with the IDE's refactoring processor to temporarily invert the extraction in the parent commit, restoring the

massive code block to its original state.²⁰ It then executes a standard textual Git merge to safely integrate the parallel logic modifications. Finally, RefMerge algorithmically replays the exact Extract Method refactoring onto the successfully merged artifact.²⁰ Empirical benchmarks across 2,001 merge scenarios confirm that RefMerge completely resolves 7% of highly complex refactoring conflicts, heavily outperforming graph-based alternatives which resolve only 4% under the same conditions.¹⁸

Optimal Industry Solution: Weave (SemanticDiff)

For CLI-driven pipelines where IDE plugins are unviable, Weave serves as the optimal industry driver. Weave addresses extraction outputs through entity-level semantic merging. Powered by the sem-core library and tree-sitter parsers, Weave extracts functions, classes, and methods as discrete semantic entities rather than raw AST nodes.²² When an Extract Method creates a new function, Weave registers it as an entirely independent entity. If two developers extract different methods or add independent logic to the same file simultaneously, Git produces false conflicts due to line proximity. Weave bypasses this by matching entities via structural hashing; because the extracted methods possess distinct entity identities (name, type, and scope), Weave auto-resolves the file, achieving a 100% clean merge rate on complex entity-collision benchmarks where Git succeeds only 48% of the time.²²

Intra-File Relocations and Signature Mutations

RefactoringMiner 2.0 outputs detailing signature modifications (Change Return Type, Change Parameter Type, Rename Parameter, Add/Reorder Parameter) and intra-file movements (Move Method, Rename Method) trigger conflicts in line-based VCS due to spatial overlap.⁴ For example, a Change Return Type on the left branch and a Change Parameter Type on the right branch target the exact same line of the method signature, resulting in a hard conflict despite being logically distinct operations.⁵

Optimal Industry Solution: Mergiraf

Mergiraf is a highly performant, syntax-aware merge driver written in Rust that supports 33 languages.⁵ It is specifically optimized for signature mutations and intra-file overlap conflicts. Mergiraf parses the target files into generic syntax trees and employs the GumTree algorithm to establish fuzzy matches between non-conflicting subtrees, ultimately converting the trees into Parent-Child-Sibling (PCS) triples.⁵

When analyzing a method signature, Mergiraf treats the return type, the method identifier, and the parameter list as separate, non-overlapping AST regions.⁵ Therefore, parallel modifications to the return type and the parameters are integrated cleanly. Furthermore, Mergiraf incorporates language-specific rules defining commutative parents. While it permits non-overlapping signature changes, it rigorously checks for duplicate key generation or syntax violations—such as two branches independently adding fields with identical names but differing types—and isolates the conflict markers precisely around the offending leaf nodes rather than invalidating the entire block.⁵

Field, Attribute, and Hierarchical Transformations

Refactoring operations that manipulate object-oriented state and inheritance structures yield outputs such as Pull Up Attribute, Push Down Attribute, Extract Attribute, and Encapsulate Attribute.³ These refactorings actively modify the structural contract of a class hierarchy, migrating state variables between superclasses and subclasses.

Optimal Research Solution: IntelliMerge

IntelliMerge is a graph-based, refactoring-aware merging algorithm optimized explicitly for field-level and hierarchical object-oriented conflicts.²⁶ IntelliMerge translates the Java source code into Program Element Graphs (PEGs).²⁸ Within a PEG, nodes represent classes, methods, and fields, while the edges define strict hierarchical dependencies, such as "class implements interface" or "class defines field".²⁷

When RefactoringMiner 2.0 outputs a Pull Up Attribute, moving a field from a subclass to an abstract parent class, IntelliMerge executes consecutive top-down and bottom-up matching phases across the PEGs.²⁷ This dual-phase graph matching allows IntelliMerge to definitively trace the relocation of the attribute through the inheritance tree. Qualitative benchmarking demonstrates that while IntelliMerge struggles with large-scale cross-file Move Class operations, it is exceptionally effective at handling ordering, formatting, and field-level relational conflicts, reducing overall merge conflicts by 58.9% compared to unstructured Git merging.²⁶

Sub-Method, Variable, and Conditional Logic Transformations

The distinguishing feature of RefactoringMiner 2.0 over its predecessor is its capability to detect sub-method level refactorings.¹ These transformations include Extract Variable, Inline Variable, Rename Variable, Split Conditional, Invert Condition, and Merge Catch.⁴ Because these operations occur entirely within the AST boundaries of a single method body, entity-level tools (which treat method bodies as opaque blocks) and graph-based tools are fundamentally ill-equipped to resolve them.²² Full AST node-level syntax merging is mandatory.

Optimal Research Solution: Spork (For Java)

For Java codebases, Spork is the definitive merge tool for sub-method refactorings. Spork utilizes the Spoon library to parse Java source files into highly detailed ASTs and leverages a modified 3DM algorithm to detect and resolve intra-method conflicts.³⁰ The primary drawback of historical AST-based merge tools is the destruction of developer formatting during the tree-rebuilding phase. Spork bypasses this by meticulously identifying indentation styles and caching original source code tokens directly from the Spoon parser.³⁰ When sub-method refactorings like Extract Variable are resolved, Spork reuses these original tokens during the pretty-printing phase, successfully preserving developer formatting in over 90% of merged files and executing significantly faster than legacy tools like JDime.³¹

Optimal Industry Solution: Mergiraf (Multi-Language)

For environments operating outside of Java, Mergiraf remains the optimal tool for resolving conditional and variable-level modifications. When dealing with an Invert Condition or Rename Variable refactoring, Mergiraf first attempts a highly optimized line-based merge.⁵ If a conflict occurs, it marks all successfully merged syntax subtrees as resolved, drastically reducing the search space.⁵ It then isolates the precise leaf nodes representing the variables or conditional operators involved in the conflict, allowing it to weave logical variable renames seamlessly into parallel logic updates without generating syntax errors.⁵

The Impact of Autonomous AI Agents on Refactoring Merge Strategies

The necessity for precise refactoring-aware merge tooling has compounded rapidly due to the proliferation of autonomous AI coding agents (e.g., Claude Code, Cursor Bugbot, Qodo).³³ Machine learning models operating in parallel worktrees frequently execute localized, sub-method refactorings or method extractions to optimize their context windows and adhere to output token limits.³⁵

When these agents attempt to merge their modifications back into a main branch, line-based version control systems trigger massive false-positive conflicts because multiple agents have independently added or extracted different functions within the same file.²² To alleviate this, entity-level tools like Weave are deployed directly as Model Context Protocol (MCP) servers.³⁵ This integration allows AI agents to interface directly with the merge driver's parsing engine, claim specific semantic entities prior to code modification, and detect refactoring collisions dynamically before initiating a merge.³⁵ Transitioning to syntax-aware (Mergiraf) and entity-level (Weave) merge drivers ensures that the continuous, high-velocity refactorings generated by AI pipelines are successfully integrated without requiring constant manual conflict mediation.⁵

Works cited

1. RefactoringMiner 2.0 - IEEE Computer Society, accessed April 23, 2026, <https://www.computer.org/csdl/journal/ts/2022/03/09136878/1liqhds60s8>
2. RefactoringMiner 2.0 - Concordia University, accessed April 23, 2026, https://users.encs.concordia.ca/~nikolaos/publications/TSE_2020.pdf
3. osmarleandro/RefactoringMiner: Forked repo. See the original - GitHub, accessed April 23, 2026, <https://github.com/osmarleandro/RefactoringMiner>
4. RefactoringMiner/documentation/history.md at master - GitHub, accessed April 23, 2026, <https://github.com/tsantalis/RefactoringMiner/blob/master/documentation/history.md>
5. Mergiraf: syntax-aware merging for Git - LWN.net, accessed April 23, 2026, <https://lwn.net/Articles/1042355/>
6. ConflictBench: A Benchmark to Evaluate Software Merge Tools - People,

- accessed April 23, 2026,
<https://people.cs.vt.edu/nm8247/publications/bowen-jss-2024-preprint.pdf>
7. tsantalis/RefactoringMiner - GitHub, accessed April 23, 2026,
<https://github.com/tsantalis/RefactoringMiner>
 8. Refactoring-Aware Patch Integration Across Structurally Divergent Java Forks - arXiv, accessed April 23, 2026, <https://arxiv.org/pdf/2508.06718>
 9. Accurate Abstract Syntax Tree Differencing: Language-Aware Design, Benchmarking, and Empirical Assessment - Concordia's Spectrum, accessed April 23, 2026,
https://spectrum.library.concordia.ca/996326/1/Alikhanifard_PhD_S2026.pdf
 10. Plastic SCM Reviews in 2026 - SourceForge, accessed April 23, 2026,
<https://sourceforge.net/software/product/Plastic-SCM/>
 11. Plastic SCM in a nutshell - by Çağatay ÇELİK - Medium, accessed April 23, 2026,
<https://medium.com/xperexpo/plastic-scm-in-a-nutshell-326007317eaf>
 12. A Merge Tool that Understands Functions - InfoQ, accessed April 23, 2026,
<https://www.infoq.com/news/2013/04/Semantic-Merge/>
 13. Merge Smarter With Semantic Merge - C# Corner, accessed April 23, 2026,
<https://www.c-sharpcorner.com/UploadFile/36985e/merge-smarter-with-semantic-merge/>
 14. Assembler Code Refactoring vs. Plastic SCM Comparison - SourceForge, accessed April 23, 2026,
<https://sourceforge.net/software/compare/Assembler-Code-Refactoring-vs-Plastic-SCM/>
 15. Refactoring-Aware Patch Integration Across Structurally Divergent Java Forks - IEEE Xplore, accessed April 23, 2026,
<https://ieeexplore.ieee.org/document/11190222/>
 16. How we refactor and how we document it? On the use of supervised machine learning algorithms to classify refactoring documentati, accessed April 23, 2026,
https://repository.rit.edu/context/article/article/3100/viewcontent/ASOC_20_How_We_Refactor_and_How_We_Document_it_.pdf
 17. ActRef: Enhancing the Understanding of Python Code Refactoring with Action-Based Analysis - arXiv, accessed April 23, 2026,
<https://arxiv.org/html/2505.06553v1>
 18. A Systematic Comparison of Two Refactoring-aware Merging Techniques Max Ellis, accessed April 23, 2026,
<https://ualberta.scholaris.ca/bitstreams/b0efc791-0bb4-4672-8be4-055898ddce4c/download>
 19. Operation-based Refactoring-aware Merging: An Empirical Evaluation - Sarah Nadi, accessed April 23, 2026,
<https://sarahnadi.org/assets/pdf/pubs/EllisTSE2023.pdf>
 20. ualberta-smr/RefMerge: Refactoring-aware Merging IntelliJ plugin - GitHub, accessed April 23, 2026, <https://github.com/ualberta-smr/RefMerge>
 21. A Universal Textual Merge Strategy Based on Tokens for Version Control Systems - arXiv, accessed April 23, 2026, <https://arxiv.org/html/2604.13813v1>
 22. GitHub - Ataraxy-Labs/weave: Entity-level semantic merge driver for Git. Resolves

- conflicts that git can't by understanding code structure via tree-sitter. 31/31 clean merges vs git's 15/31., accessed April 23, 2026, <https://github.com/ataraxy-labs/weave>
23. Supporting additional structured merge tools — entity-level merge as complement to Mergiraf · jj-vcs jj · Discussion #8831 - GitHub, accessed April 23, 2026, <https://github.com/jj-vcs/jj/discussions/8831>
 24. Change Signature refactoring | JetBrains Rider Documentation, accessed April 23, 2026, https://www.jetbrains.com/help/rider/Refactorings_Change_Signature.html
 25. Architecture - Mergiraf, accessed April 23, 2026, <https://mergiraf.org/architecture.html>
 26. Evaluation of Version Control Merge Tools | Request PDF - ResearchGate, accessed April 23, 2026, https://www.researchgate.net/publication/385286884_Evaluation_of_Version_Control_Merge_Tools
 27. (PDF) IntelliMerge: a refactoring-aware software merging technique - ResearchGate, accessed April 23, 2026, https://www.researchgate.net/publication/336457962_IntelliMerge_a_refactoring-aware_software_merging_technique
 28. Evaluation of Version Control Merge Tools - University of Washington, accessed April 23, 2026, <https://homes.cs.washington.edu/~mernst/pubs/merge-evaluation-ase2024.pdf>
 29. IntelliMerge: A Refactoring-Aware Software Merging Technique (SPLASH 2019 - OOPSLA), accessed April 23, 2026, <https://2019.splashcon.org/details/splash-2019-oopsla/53/IntelliMerge-A-Refactoring-Aware-Software-Merging-Technique>
 30. Spork: Structured Merge for Java With Formatting Preservation - IEEE Computer Society, accessed April 23, 2026, <https://www.computer.org/csdl/journal/ts/2023/01/09684709/1AgmpjY15T2>
 31. Investigating the Merge Conflict Life-Cycle Taking the Social Dimension into Account, accessed April 23, 2026, <https://www.se.cs.uni-saarland.de/theses/GustavoDoValeDiss.pdf>
 32. Basic rules of three-way merge. | Download Table - ResearchGate, accessed April 23, 2026, https://www.researchgate.net/figure/Basic-rules-of-three-way-merge_tbl1_328508000
 33. AI Code Review Automation: Complete Guide 2025 - Digital Applied, accessed April 23, 2026, <https://www.digitalapplied.com/blog/ai-code-review-automation-guide-2025>
 34. State of AI Code Review Tools in 2025 - DevTools Academy, accessed April 23, 2026, <https://www.devtoolsacademy.com/blog/state-of-ai-code-review-tools-2025/>
 35. Weave – A language aware merge algorithm based on entities | Hacker News, accessed April 23, 2026, <https://news.ycombinator.com/item?id=47241976>
 36. Entity-level merge for multi-agent parallel edits · Issue #13554 · openai/codex - GitHub, accessed April 23, 2026, <https://github.com/openai/codex/issues/13554>